

Gradient Descent (From Newton to Adam)

Consider an eqn $f(x) = 0$.

In high school math, we find the exact solution x^* that satisfies $f(x^*) = 0$.

ex) $x^2 - 5x + 6 = 0 \Rightarrow x = 2 \text{ or } 3$.

However, it's extremely hard to find the exact soln for a "general" eqn $f(x) = 0$.

How about finding x s.t. $f(x) \approx 0$?

$\Rightarrow$ Numerical methods.

Each numerical method (iterative method) requires three components.

① Initial point.

② Updating rule

③ Stopping rule.

GD is also an iterative alg. for minimizing the objective fcn.

To understand this, we start from the basic numerical methods.

Note that each method has pros and cons!

I. Root-finding methods.

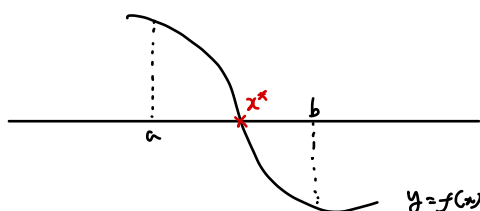
Goal Solving $f(x) = 0$.

1. Bisection Method.

Assumption f is continuous and the solution exists.

Key Idea

Let $a < b$. If $f(a)f(b) < 0$, then $\exists x^* \in (a, b)$ s.t. $f(x^*) = 0$. (Intermediate Value Theorem)



Algorithm

① Initial: $a_0 < b_0$ w/ $f(a_0)f(b_0) < 0$

② Let $m_k = \frac{a_k + b_k}{2}$

$\left\{ \begin{array}{ll} \text{If } f(m_k)f(b_k) < 0, \text{ then } & \begin{array}{l} a_{k+1} = m_k \\ b_{k+1} = b_k \end{array} \\ \text{If } f(m_k)f(a_k) < 0, \text{ then } & \begin{array}{l} a_{k+1} = a_k \\ b_{k+1} = m_k \end{array} \\ \text{If } f(m_k) = 0 \rightarrow \text{stop.} \end{array} \right.$

③ If $b_k - a_k < \varepsilon$, for some small $\varepsilon > 0$, then we stop.

or
 $|f(a_k)| < \varepsilon$ or $|f(b_k)| < \varepsilon$

⊕ • The interval $[a_k, b_k]$ always contains a root.

• $b_k - a_k = \frac{1}{2^k} (b_0 - a_0)$.

This means that the error is halved each step!

⊖ • Only uses the sign of f . What if we can access more info of f ?

(Discussion)

2. Newton's Method.

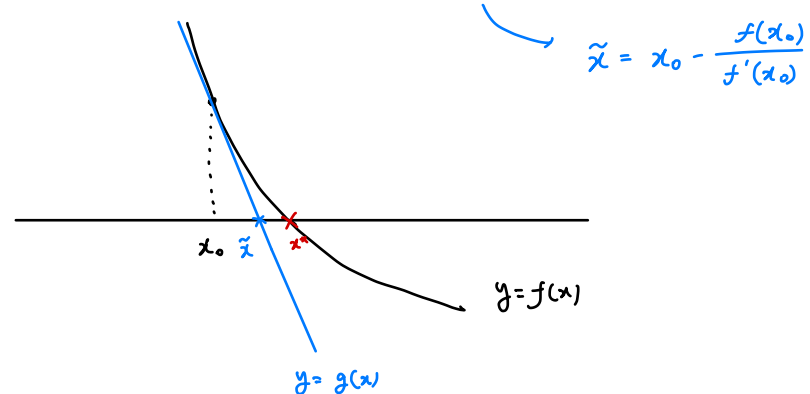
Assumption f is differentiable and the solution exists.

Key Idea

Linear (1st order) approximation of differentiable function (Taylor Theorem)

Around $x = x_0$, $f(x) \approx g(x) = f(x_0) + f'(x_0)(x - x_0)$.

Thus, we expect $x^* \approx \tilde{x}$, where $f(x^*) = 0$
 $g(\tilde{x}) = 0$. → linear approximation of f at x_0 .



Alg

① Initial : any guess x_0

② $x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$

③ If $x_{k+1} - x_k < \varepsilon$, for some small $\varepsilon > 0$, then we stop.
or

$f(x_k) < \varepsilon$.

⊕ • Faster than Bisection for some cases.
(makes sense since we use more info)

⊖ (Discussion)

II. Optimization

Goal Find $\operatorname{Argmin}_{x \in \mathbb{R}} f(x)$.

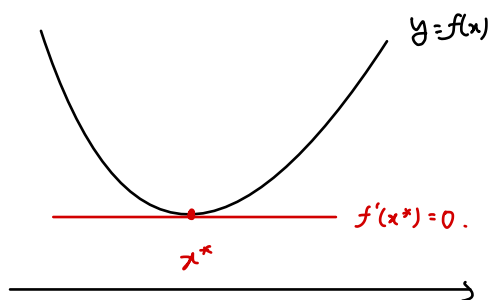
1. Gradient Descent

Assumption • f is convex and differentiable.

• $x^* = \operatorname{Argmin}_{x \in \mathbb{R}} f(x)$ exists.

Key Idea

x^* satisfies $f'(x^*) = 0$.



$\Rightarrow$ We can use Newton's method w/ $f'(x) = 0$.

This gives the following updating rule:

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}.$$

For univariate case, this is okay.

However, for multivariate case, computing the 2nd derivative (Hessian) is difficult in general.

Therefore, we simplify the updating rule by

$$x_{k+1} = x_k - \eta \cdot f'(x_k),$$

Where $\eta > 0$ is some fixed constant.

$\hookrightarrow$ called "step size" or "learning rate".

Alg

① Initial: any guess x_0

② $x_{k+1} = x_k - \eta f'(x_k)$.

③ If $x_{k+1} - x_k < \epsilon$, for some small $\epsilon > 0$, then we stop.

$$\begin{array}{c} \text{or} \\ |f(x_{k+1}) - f(x_k)| < \epsilon \\ \text{or} \\ \vdots \end{array}$$

⊕ • cheap computational cost.

⊖

• η is important.

• If $f'(x) \approx 0$, update is too slow.

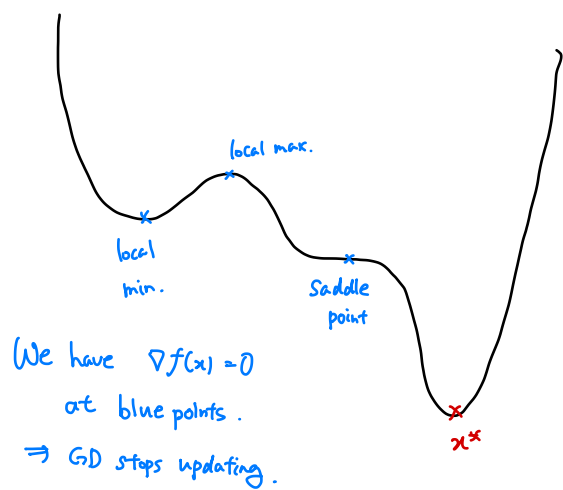
2. Adam (Adaptive momentum)

In ML, we often deal w/ non-convex multivariate f .

What are the problems of GD for such f ?

$$x_{k+1} = x_k - \eta \cdot \nabla f(x_k)$$

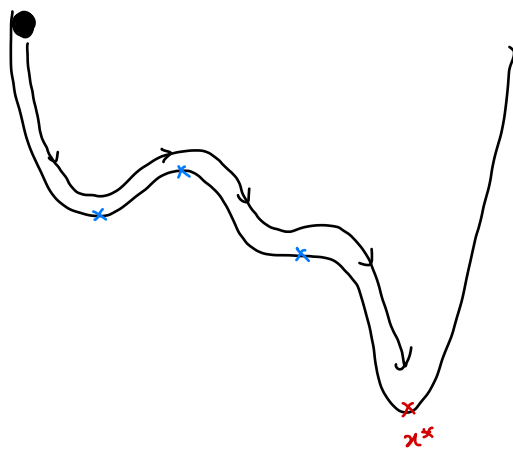
- Non-convexity



Idea Suppose that we drop a ball.

(velocity)

Then, due to the momentum,
it can go through all blue points
and arrive at x^* .



$\Rightarrow$ Add some momentum terms!

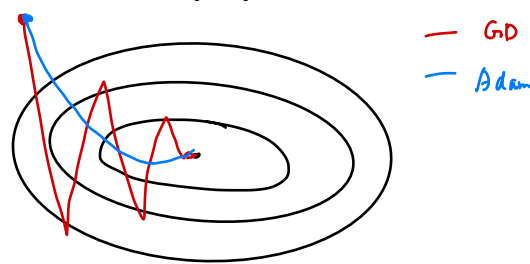
$$x_{k+1} = x_k - \eta \nabla f(x_k) + m_k$$

- Multivariate (Skip if you can't understand)

The current η is applied to all directions equally.

This doesn't utilize the information of steepness.

Adam uses adaptive learning rate for each direction,
which avoids zig-zags.



$\Rightarrow$ In practice, Adam is usually better than GD.